1. Round Robin CPU scheduling (Experiment with different quantum sizes for RR algorithm)

```c
#include <stdio.h>
#include <stdlib.h>

struct process {
    int pid, at, bt, ct, tat, wt, rt;
};

struct Queue {
    int items[100];
    int front, rear;
};

void enqueue(struct Queue *q, int value) {
    q->items[++q->rear] = value;
}

int dequeue(struct Queue *q) {
    return q->items[++q->front];
}

int isEmpty(struct Queue *q) {
    return q->front == q->rear;
}

void roundRobin(struct process p[], int n, int tq) {
    struct Queue q;
    q.front = q.rear = -1;

    int time = 0, completed = 0;
    int sum_tat = 0, sum_wt = 0;
    int visited[n];
    int gantt_pid[100], gantt_start[100], gantt_end[100], gantt_count = 0;

    for (int i = 0; i < n; i++) visited[i] = 0;
    for (int i = 0; i < n; i++) {
        if (p[i].at == 0) {
            enqueue(&q, i);
            visited[i] = 1;
        }
    }

    while (completed < n) {
        if (!isEmpty(&q)) {
            int idx = dequeue(&q);
            gantt_pid[gantt_count] = p[idx].pid;
            gantt_start[gantt_count] = time;
```

```
        if (p[idx].rt > tq) {
            time += tq;
            p[idx].rt -= tq;
        } else {
            time += p[idx].rt;
            p[idx].rt = 0;
            p[idx].ct = time;
            completed++;
        }

        gantt_end[gantt_count] = time;
        gantt_count++;

        for (int i = 0; i < n; i++) {
            if (p[i].at <= time && !visited[i]) {
                enqueue(&q, i);
                visited[i] = 1;
            }
        }

        if (p[idx].rt > 0) {
            enqueue(&q, idx);
        }
    } else {
        time++;
        for (int i = 0; i < n; i++) {
            if (p[i].at <= time && !visited[i]) {
                enqueue(&q, i);
                visited[i] = 1;
            }
        }
    }
}

printf("\nProcess Table:\n");
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
for (int i = 0; i < n; i++) {
    p[i].tat = p[i].ct - p[i].at;
    p[i].wt = p[i].tat - p[i].bt;
    sum_tat += p[i].tat;
    sum_wt += p[i].wt;
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",
           p[i].pid, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt);
}

double avgtat = (double)sum_tat / n;
double avgwt = (double)sum_wt / n;
```

```c
    printf("Average TAT = %.2f\n", avgtat);
    printf("Average WT = %.2f\n", avgwt);

    printf("\nGantt Chart:\n");
    for (int i = 0; i < gantt_count; i++) {
        printf("| P%d (%d-%d) ", gantt_pid[i], gantt_start[i], gantt_end[i]);
    }
    printf("|\n");
}

int main() {
    int n, tq;
    printf("Enter the number of processes: ");
    scanf("%d", &n);
    printf("Enter the time quantum: ");
    scanf("%d", &tq);

    struct process p[n];
    for (int i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("Enter arrival time for process %d: ", i + 1);
        scanf("%d", &p[i].at);
        printf("Enter burst time for process %d: ", i + 1);
        scanf("%d", &p[i].bt);
        p[i].rt = p[i].bt;
    }

    roundRobin(p, n, tq);
    return 0;
}
```

Output:

```
Enter the number of processes: 4
Enter the time quantum: 2
Enter arrival time for process 1: 0
Enter burst time for process 1: 5
Enter arrival time for process 2: 1
Enter burst time for process 2: 4
Enter arrival time for process 3: 2
Enter burst time for process 3: 2
Enter arrival time for process 4: 4
Enter burst time for process 4: 1

Process Table:
PID     AT      BT      CT      TAT     WT
1       0       5       12      12      7
2       1       4       11      10      6
3       2       2       6       4       2
4       4       1       9       5       4
Average TAT = 7.75
Average WT = 4.75

Gantt Chart:
| P1 (0-2) | P2 (2-4) | P3 (4-6) | P1 (6-8) | P4 (8-9) | P2 (9-11) | P1 (11-12) |
```

```
Enter the number of processes: 4
Enter the time quantum: 3
Enter arrival time for process 1: 0
Enter burst time for process 1: 5
Enter arrival time for process 2: 1
Enter burst time for process 2: 4
Enter arrival time for process 3: 2
Enter burst time for process 3: 2
Enter arrival time for process 4: 4
Enter burst time for process 4: 1

Process Table:
PID     AT      BT      CT      TAT     WT
1       0       5       10      10      5
2       1       4       12      11      7
3       2       2       8       6       4
4       4       1       11      7       6
Average TAT = 8.50
Average WT = 5.50

Gantt Chart:
| P1 (0-3) | P2 (3-6) | P3 (6-8) | P1 (8-10) | P4 (10-11) | P2 (11-12) |
```

2. Write a C program to simulate multi-level queue schedulingalgorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

```c
#include <stdio.h>

#define MAX 100

struct process {
    int pid, at, bt, type, ct, tat, wt, start;
};

void sortByArrival(struct process q[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (q[i].at > q[j].at) {
                struct process temp = q[i];
                q[i] = q[j];
                q[j] = temp;
            }
        }
    }
}

int executeQueue(struct process q[], int n, int time) {
    for (int i = 0; i < n; i++) {
        if (time < q[i].at) time = q[i].at;
        q[i].start = time;
        time += q[i].bt;
        q[i].ct = time;
        q[i].tat = q[i].ct - q[i].at;
        q[i].wt = q[i].tat - q[i].bt;
    }
    return time;
}

void printGanttChart(struct process sysQ[], int sysCount, struct process
userQ[], int userCount) {
    printf("\nGantt Chart:\n");
    for (int i = 0; i < sysCount; i++) {
        printf("| P%d (%d-%d) ", sysQ[i].pid, sysQ[i].start, sysQ[i].ct);
    }
    for (int i = 0; i < userCount; i++) {
        printf("| P%d (%d-%d) ", userQ[i].pid, userQ[i].start, userQ[i].ct);
    }
    printf("|\n");
}
```

```c
int main() {
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    struct process p[MAX], systemQ[MAX], userQ[MAX];
    int sysCount = 0, userCount = 0;

    printf("Enter PID AT BT TYPE (0=System,1=User):\n");
    for (int i = 0; i < n; i++) {
        scanf("%d %d %d %d", &p[i].pid, &p[i].at, &p[i].bt, &p[i].type);
        if (p[i].type == 0) systemQ[sysCount++] = p[i];
        else userQ[userCount++] = p[i];
    }

    sortByArrival(systemQ, sysCount);
    sortByArrival(userQ, userCount);

    int time = 0;
    time = executeQueue(systemQ, sysCount, time);
    time = executeQueue(userQ, userCount, time);

    printf("\nProcess Table:\n");
    printf("PID\tAT\tBT\tTYPE\tCT\tTAT\tWT\n");
    for (int i = 0; i < sysCount; i++) {
        printf("%d\t%d\t%d\tSys\t%d\t%d\t%d\n",
               systemQ[i].pid, systemQ[i].at, systemQ[i].bt,
               systemQ[i].ct, systemQ[i].tat, systemQ[i].wt);
    }
    for (int i = 0; i < userCount; i++) {
        printf("%d\t%d\t%d\tUser\t%d\t%d\t%d\n",
               userQ[i].pid, userQ[i].at, userQ[i].bt,
               userQ[i].ct, userQ[i].tat, userQ[i].wt);
    }

    printGanttChart(systemQ, sysCount, userQ, userCount);

    return 0;
}
```

Output:

```
Enter number of processes: 6
Enter PID AT BT TYPE (0=System,1=User):
1 5 5 0
2 4 6 1
3 3 7 0
4 1 9 1
5 2 2 0
6 6 3 1

Process Table:
PID     AT      BT      TYPE    CT      TAT     WT
5       2       2       Sys     4       2       0
3       3       7       Sys     11      8       1
1       5       5       Sys     16      11      6
4       1       9       User    25      24      15
2       4       6       User    31      27      21
6       6       3       User    34      28      25

Gantt Chart:
| P5 (2-4) | P3 (4-11) | P1 (11-16) | P4 (16-25) | P2 (25-31) | P6 (31-34) |
```